REALBIM: A Framework for Safe Early Reuse of Shared GPU Buffers Across Processes

Xingbo Hui[1], Xuetao Jiang[2], Guodong Ye[1], Ruochen Shao[1], Tianyi Wang[1], Sicheng Wu[1], Rui Zhou[1,*], Qingguo Zhou[1,*]
[1]School of Information Science and Engineering, Lanzhou University, Lanzhou, China
[2]School of Artificial Intelligence, Zibo Polytechnic University, Zibo, China
[*]Co-corresponding authors: zr@lzu.edu.cn, zhouqg@lzu.edu.cn

***Abstract*—Autonomous-driving and robotics software runs as many processes, several of which read the same large GPU tensor, such as a bird's-eye-view (BEV) feature map. Sharing that tensor instead of copying it saves memory and bandwidth, but the producer must then decide when the next frame may overwrite it. Overwriting it once submitted GPU work finishes can feed the wrong frame to a process that has not started reading, and holding it until every result is ready drops frames while the GPU idles. Our key observation is that a shared tensor has two lifetimes: its storage is needed only until the last operation that reads it, but its frame identity must last until every result is delivered. REALBIM builds on this split. Its cross-process layer, Borrow-in-Memory (BIM), protects every recipient from the moment a frame is published. Its in-process layer, ReadSeal, finds each recipient's last read by alias analysis and refuses to run if the code, model, or input layout has changed. On an A100 at 25–40 Hz, with recipients whose models stop reading the map early, releasing at the last read delivers 1.31–1.50× the timely results of holding the buffer until all results are ready, at 8–11 ms more mean latency. It also matches per-recipient copies without their 256 MiB of extra memory. REALBIM derives that point automatically. In controlled tests where one reader finished before another had submitted its reads, releasing once submitted work completed let the second reader compute on the next frame. REALBIM kept it on the correct frame and refused all 22 plans run against changed code, model state, or inputs.**

***Index Terms*—GPU buffer reuse, zero-copy IPC, buffer lifetime, alias analysis, autonomous driving.**

I. Introduction

Autonomous-driving stacks such as Autoware run as many processes on ROS 2, the Robot Operating System [1]–[3], and several of them consume the same data (Fig. 1). For example, a camera–LiDAR fusion node publishes a bird's-eye-view (BEV) feature map, a top-down grid of features around the vehicle. Separate processes for 3D detection, tracking, motion planning, and data recording read the map on their own schedules. We call each such process a *recipient*, and each component inside a recipient that reads the map a *reader*. A recipient can have several readers: BEVFusion, for instance, feeds one BEV map to both a detection and a map-segmentation head [4].

Copying the map for each recipient is expensive. Our evaluated map is a $1\times512\times180\times180$ fp32 tensor of 63.3 MiB [4]. Private copies for four recipients would add 256 MiB of device memory (64 MiB each after allocator rounding), and at 30 Hz the copying alone would read and write about 15 GiB/s. On an automotive system-on-chip (SoC), whose GPU shares DRAM

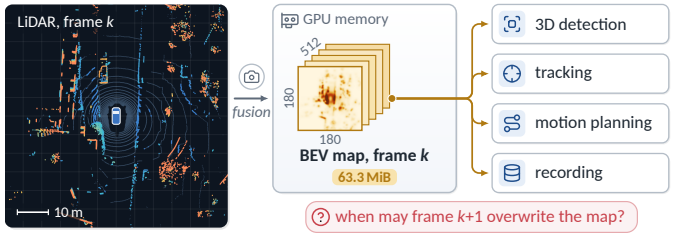


Fig. 1. **One BEV map, four recipients.** Fusing each LiDAR sweep (left, from nuScenes [5]) with camera images yields a 63.3 MiB map in GPU memory; its front face sketches one of 512 feature channels. Sharing the map avoids four copies, but the producer must decide when frame $k{+}1$ may overwrite it.

with the rest of the stack [6], that traffic is about 8% of an AGX Orin-class module's 204.8 GB/s [7], and both costs grow with every recipient. Zero-copy inter-process communication (IPC) avoids them by letting recipients map the producer's buffer [8], [9], and recent work and ROS 2 facilities extend this to GPU memory [10], [11]. The producer must then decide when it may overwrite that buffer, which we call a *slot*.

Two common release policies err in opposite directions (Section II). GPU work is asynchronous: a process can wait only for GPU work that has already been submitted. *Stream-only release* therefore reuses the slot once all GPU work submitted so far has finished. A recipient still waiting in its queue has submitted nothing, so it later computes on the next frame, and nothing flags the mistake. A detector would then report objects seen in frame $k{+}1$ under the timestamp of frame k; at 20 Hz, a vehicle driving at 20 m/s moves a full meter between the two. *Full retention* holds the slot until every result is ready, as reference-counted zero-copy transports do when recipients keep their references until done [8], [12]. With one slot, frames that arrive during the hold are dropped. In our 20 Hz trace the hold barely exceeds the frame period, so full retention drops every other frame and leaves the GPU idle in between (Fig. 4). A second slot recovers most of these frames but costs another 64 MiB per shared tensor; we show that one slot, released at the right moment, delivers as many results as two (Section V-C).

Our key observation is that the tensor in a slot, which we call the *source*, has two lifetimes (Fig. 2). Its *storage lifetime* ends when the last operation that reads it completes. Its *identity lifetime* lasts until every result is delivered, because those results must still name the frame they came from, much as an

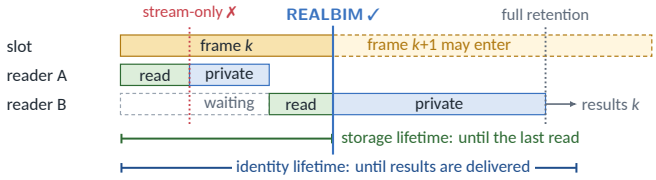


Fig. 2. **Two lifetimes of one frame** in a recipient with readers A and B. Storage is needed until B's read, the last one; identity k, until results are delivered. Stream-only release frees the slot before B reads; REALBIM frees it at the last read, so frame $k{+}1$ can enter while B computes.

essay keeps citing a library book after the book has gone back to the shelf. The two can differ widely. In the TransFusion detection head that reads our BEV map, only the first of 287 operations reads the source; the rest compute on intermediate results in private memory and are unaffected when the source is overwritten. This head is not an outlier: each of the nine torchvision model tails that our analysis accepts stops reading its input within its first 12 operations (Section V-F). A recipient therefore needs its *borrow*, its permission to read the slot, only until its last source read completes. Full retention, copying, and early reuse then differ only in where the borrow ends: when the outputs are complete, when a private copy is complete, or at the last source read. All three must protect a recipient from the moment its frame is published, before it starts reading.

Ending the borrow at the last read is safe only if three questions can be answered. (1) Which operation reads last? *Views*, tensors that share the source's memory, extend its reads, so this *read boundary* can come long after the source itself last appears. It is also easy to misplace by hand. In a ResNet-18 tail [13], a residual connection reads the source again at operation 12 (Fig. 5), so a boundary placed after the first read would free the source too soon. (2) Who may still read? A reader that has not yet submitted GPU work leaves no event to wait on. (3) Is the analyzed program the one that runs? Adding one read near our detection head's end moves its boundary from the first operation to the second-to-last (Fig. 5), so a boundary derived for one model version can be wrong for the next.

We present REALBIM (REAd-Last BIM), a framework that answers these questions in two layers split at the process boundary (Fig. 3). The producer knows only which processes receive a frame, so the cross-process layer, Borrow-in-Memory (BIM), answers question (2). It registers a borrow for every recipient when the frame is published, before any recipient starts (Section III). Each recipient knows its own components, so the in-process layer, ReadSeal, answers questions (1) and (3). It derives each reader's boundary from its model graph and refuses a plan that no longer matches the program (Section IV).

With recipients whose last read comes early, on an A100 at 25–40 Hz, REALBIM delivers 1.31–1.50× the *timely* results (correct and within 100 ms) of full retention at 8–11 ms more mean latency. It matches per-recipient copies without their memory and bandwidth cost, and it never lets a reader compute on the wrong frame, as stream-only release does. After each of 16 replayed model changes, ReadSeal rebuilds the boundary in 0.08–0.21 s, where a hand-placed boundary is refused until a person moves it. Section VI discusses when a private copy is the better choice. Our contributions are:

- **Two lifetimes for shared GPU tensors**, which make full retention, copying, and early reuse one mechanism that ends a borrow at different points.
- **BIM: protection from publication** (Section III), which covers recipients that have not yet submitted any GPU work, as no event-based policy can, and ends each borrow at a GPU completion event.
- **ReadSeal: a derived, checked release point** (Section IV), which finds each reader's last read by alias analysis and refuses a plan whose code, model, or input layout has changed.

II. Background and Motivation

This section gives the GPU background and shows where existing policies end the borrow (Table I).

GPU background. A *kernel* is a function executed on the GPU. A process places kernels in a *stream*, a queue that the GPU runs in order, and continues without waiting; a read ends when its kernel completes. An *event* marks a position in a stream and completes after the preceding work. PyTorch's caching allocator reuses freed memory in stream order [14], and `record_stream` extends this to other streams by waiting for work already enqueued on them [15]. CUDA IPC lets another process map a device buffer and wait on shared events, but an event marks only work already enqueued: no process can wait on reads another has not submitted.

Stream-only release. A policy that waits only for submitted work, such as `record_stream` or a read event, releases the slot once the other recipients' reads complete, although a recipient still waiting in its queue has enqueued nothing. The same gap opens inside one recipient whose readers run on different runtimes, as when BEVFusion's detection and segmentation heads read one map [4] and one of them is compiled to an inference engine. Our running example (Fig. 3b) models this with two readers. Engine A, built with NVIDIA's TensorRT, stands in for the compiled head: it runs a convolution, ReLU, and pooling stage on the map and signals an input-consumed event once it has read its input [16]. Graph B is the detection head, exported from PyTorch, and receives the map as a view through an adapter that does not read it. So when A's event completes, B has submitted nothing and there is no event to wait on: stream-only release makes the slot reusable while B still needs it (Section V-E tests this ordering). ROS 2's CUDA buffer backend offers both choices: destroying a read handle after the engine's read gives stream-only release; holding it, full retention [11].

Copying the source. A private copy lets a recipient compute after the slot is reused, but the copy itself reads the source, so the slot must stay protected until the copy completes. Our *clone-on-accept* baseline copies the source at acceptance, relies on BIM until then, and ends each borrow when the copy completes.

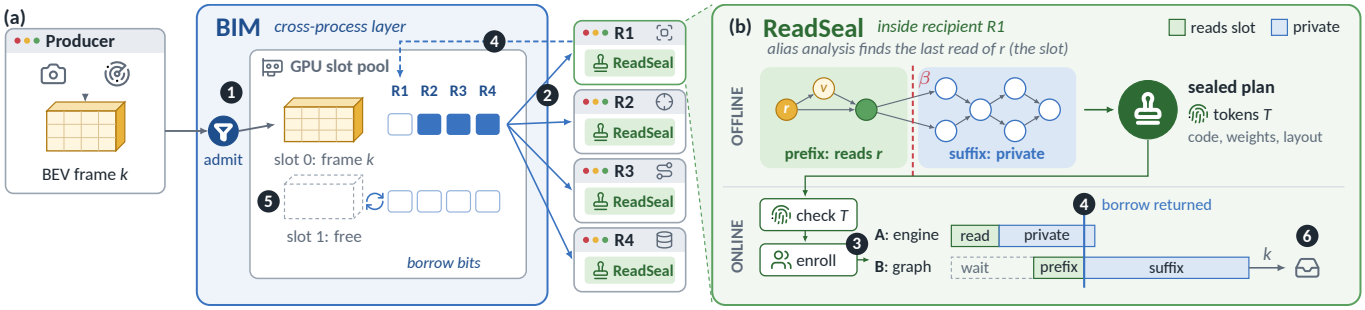


Fig. 3. **REALBIM's two layers.** (a) BIM, in the producer and shared memory, keeps a pool of GPU slots with one borrow bit per recipient and slot. (b) ReadSeal, inside each recipient: offline, alias analysis splits a reader's graph at its last read β of source r into a prefix that reads r and a private suffix, and tokens T bind the plan to code, weights, and input layout; online, the recipient checks T (also at each stage entry) and enrolls readers A, a TensorRT engine, and B, an exported graph. Numbers follow frame k (Section III).

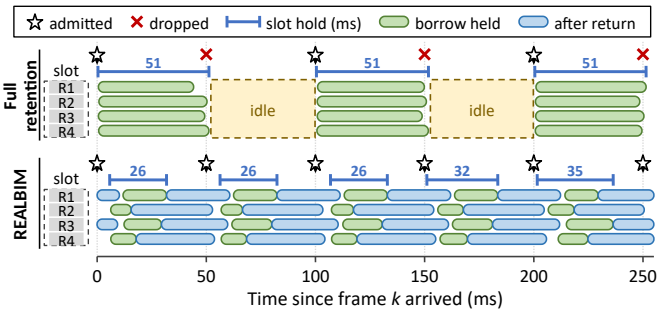


Fig. 4. **Full retention refuses frames while the GPU idles.** Measured one-slot trace at 20 Hz; R1–R4 are the recipients, and labels give slot hold times in ms. Full retention's 51 ms hold just exceeds the 50 ms period, so every other frame is refused. Under REALBIM, each recipient's work after its borrow return overlaps the next frame.

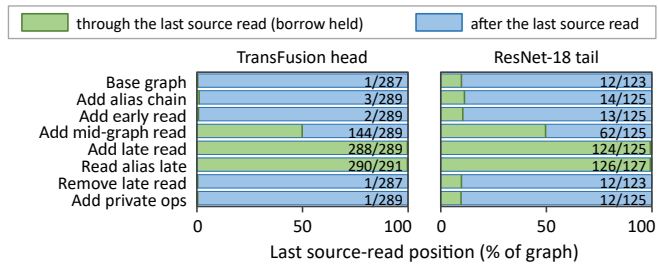


Fig. 5. **Model edits move the last source read.** Each bar is a graph snapshot that applies one edit to the base graph (Remove late read undoes Add late read). Green marks the operations up to the last source read, during which the recipient holds its borrow; labels give that operation's index / total operations.

III. The REALBIM Framework and Its BIM Layer

REALBIM tracks the two lifetimes separately (Fig. 3). A frame identity, attached when the frame is written and carried by every result, covers the identity lifetime; per-recipient borrows cover the storage lifetime. BIM lives in the producer, which owns a pool of N reusable GPU slots on one machine, and in shared memory; ReadSeal (Section IV) runs inside each recipient and decides when its borrow ends. The layers meet at one borrow bit per recipient and slot, which BIM sets before the recipient can read and ReadSeal clears after its last source read. One borrow covers all of a recipient's readers, even those yet to submit work. A recipient *returns* its borrow by clearing its bit; the slot becomes reusable when its last bit clears.

TABLE I
Release policies for four recipients. Extra memory counts input copies beyond the shared slot.

Policy	Extra memory	Slot reusable after	Read boundary	Wrong frame?
Stream-only	—	submitted reads	—	possible
Full retention	—	all outputs	—	no
Clone-on-accept	256 MiB	all copies	the copy	no
Hand-placed	—	all last reads	by hand	no
REALBIM	—	all last reads	derived	no

Life of a frame. The numbers in Fig. 3 follow frame k. BIM's producer *admits* it into a free slot (❶), *commits* it by setting every recipient's borrow bit, and then sends each recipient a small descriptor (❷). The descriptor carries the frame identity (producer instance and sequence number), a storage locator, and the layout to read. In each recipient, ReadSeal *accepts* the request, checks its plan, and *enrolls* the readers (❸); after the last source read, the recipient returns its borrow (❹). A slot whose bits are all clear takes the next frame (❺), while results labeled k are still being delivered (❻). Descriptors travel over a control plane such as ROS 2, recipients map the tensor through PyTorch's CUDA IPC support [14], and BIM rejects stale identities and incompatible layouts.

Borrow bits and admission. A process-shared lock protects slot reservation, the frame sequence, and one borrow bit per recipient, set while that recipient may still read the slot. An arriving frame takes a slot whose bits are all clear; if none is free, the producer drops the frame instead of waiting. Once the frame is written, the producer commits it by setting every recipient's bit before sending any descriptor. A recipient still waiting in its queue is therefore protected, because its bit was set before it could possibly read. Clearing the last bit makes the slot reusable without deallocating or unmapping it. If frame k is committed at time c_k and recipient j returns its borrow at time $t_{j,k}$, the slot is held for H_k. With one slot, a frame arriving at time a is admitted only if

$$a \geq c_k + H_k, \qquad H_k = \max_j t_{j,k} - c_k. \tag{1}$$

Algorithm 1 REALBIM per frame: BIM's publication and ReadSeal's plan and invocation. $b_{s,j}$: borrow bit of recipient j on slot s; U_j: readers of j; T: binding tokens.

procedure PUBLISH(frame k) ▷ producer (BIM)
  if no slot is free or cap K has no room for k, **drop** k
  write k into a free slot s; *commit*: set $b_{s,j}$ for all j
  send each recipient a descriptor of (k, s)
procedure BUILDPLAN(graph G) ▷ offline, per model version
  Q_r by (2), β by (3); split G at β; record U_j, T
procedure INVOKE(plan, frame k) ▷ recipient j
  check T; keep identity k for every output
  enroll every reader in U_j before any of them submits GPU work
  submit the stages in declared order, checking T at each entry
  on failed check: cancel unsubmitted reads, drain, publish nothing
  on each completed or cancelled read: **if** (4) holds, clear $b_{s,j}$
  on completed outputs: publish them with identity k

Under full retention, $t_{j,k}$ is when j's outputs complete; under REALBIM, it is when j's last source read completes (Section IV-B), and the shorter hold admits more of the arriving frames. Each admitted frame gives every recipient one *request*, which the recipient accepts when it takes the request from its queue. The request stays outstanding until its result reaches the output sink. An optional admission cap K bounds the pipeline's outstanding requests; a frame is admitted only if a slot is free and the cap allows it.

IV. The ReadSeal Layer: Checked Borrowing

ReadSeal prepares a *plan* once per model version and checks it each time a recipient processes a frame (Algorithm 1). The plan records where the graph stops reading shared storage, which components may read it, and which program version was analyzed. One execution of a plan is an *invocation*.

A. Deriving the Read Boundary

The analysis runs on each reader's exported graph [17]: a directed acyclic graph (DAG) of operations with fixed tensor shapes. Only the storage lifetime needs analysis, because every result carries the frame identity attached at acceptance. Sharing propagates through views: a slice of the source is a view, so an operation on the slice still reads the source, whereas a convolution reads the source but writes fresh storage, so operations on its output do not.

Formally, let $S(v)$ be the set of shared roots whose storage a value v may alias, with $S(r) = \{r\}$ for the source r. For an operation o with inputs v_i and output v_o, let $\mathcal{A}_o$ index the inputs the output may alias and $\mathcal{R}_o$ the inputs o reads. Then

$$\begin{aligned} S(v_o) &= \textstyle\bigcup_{i \in \mathcal{A}_o} S(v_i), \\ Q_r &= \{\, o : \exists\, i \in \mathcal{R}_o,\ r \in S(v_i) \,\}, \end{aligned} \tag{2}$$

where Q_r is the set of operations that may read the source. With the operations numbered $o_1, \ldots, o_n$ in execution order, the read boundary is the position of the last one in Q_r,

$$\beta = \max\{\, i : o_i \in Q_r \,\}, \tag{3}$$

and the *prefix* $o_1, \ldots, o_\beta$ contains every source read, while the *suffix* $o_{\beta+1}, \ldots, o_n$ computes the remaining results in private storage. If no operation reads the source, Q_r is empty and we set $\beta = 0$. Fig. 5 labels each snapshot with β/n; the base head has $\beta = 1$ and $n = 287$.

The sets $\mathcal{A}_o$ and $\mathcal{R}_o$ come from reviewed summaries of 42 operator variants in ATen (PyTorch's operator library), plus fixed-index tuple selection. Each summary conservatively counts every input as read and lets an output share storage whenever the operator's declared signature permits it. Plan construction rejects what it cannot analyze: unsupported in-place updates, calls into code it cannot inspect, and source views that leave the graph. The implementation compiles the prefix and suffix into separate executable stages, and a CUDA event after the prefix marks the end of its source reads.

B. Enrolling Readers Before Submission

BIM already protects each recipient from commit on. When the recipient accepts its request, ReadSeal *enrolls* (registers) all of its readers before any of them submits GPU work. Each reader u marks its last source read with a completion event: A's input-consumed event or the event after B's prefix. Let U_j be the readers of recipient j. Then j returns its borrow for frame k at the first moment when

$$\forall u \in U_j : \quad done_u \vee cancelled_u, \tag{4}$$

where the predicate $done_u$ becomes true when u's completion event completes, and $cancelled_u$ when u is cancelled before submission. Every enrolled reader takes part in this condition, even one that has enqueued nothing: when A's event completes, B's read is still pending, so the bit stays set. Once all recipients have returned their borrows, the producer can write the next frame while j's suffix keeps computing results labeled k.

C. Binding Plans to Execution

After a model update, a patched function or global variable, or an input with different strides, a plan's boundary can fall before a read that the running program performs. The producer would then overwrite the source while it is still in use. At construction, ReadSeal captures the functions to run and copies the model's weights and other state into plan-owned storage, so later edits by the caller cannot change a plan in use; the hand-placed baselines keep the same copy.

ReadSeal then records *binding tokens* T for everything the boundary depends on. Code and global variables define the operations. Model storage and version tell whether the model the caller is about to run is still the one the plan was built from. The input layout matters because aliasing depends on strides: a reshape returns a view only when the strides allow it. The compiled stages are what actually executes. At acceptance and at each stage entry ReadSeal compares T with the values $\hat{T}$ for the execution about to run and proceeds only if $\hat{T} = T$. A failed check cancels unsubmitted reads, drains started work, and publishes nothing. Applications supply the exported graph, each engine's last-input-read event, and component and output dependencies; new operators need reviewed summaries.

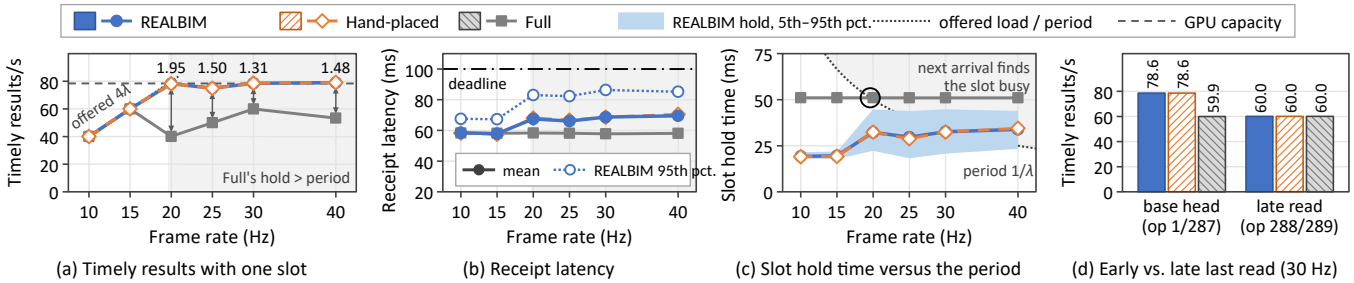


Fig. 6. **Early reuse raises one-slot delivery under load, at a latency cost.** One slot, no pipeline-wide cap K, six runs per condition and policy; each frame offers four recipient results. Hand-placed releases at the same point as REALBIM by design (Section V). (a) Labels give REALBIM/Full ratios. (b) Mean receipt latency and REALBIM's mean per-run 95th percentile. (c) Mean hold and REALBIM's 5th–95th percentile; shading marks where Full's hold exceeds the frame period. (d) Moving the last read to the graph's end removes the gain.

D. Correctness Conditions

The analysis must include every operation that can read the source. It assumes that operator summaries conservatively describe reads and storage sharing, kernels obey those summaries, and the graph executes in the analyzed order.

Proposition 1. *Under these assumptions, Q_r in Eq. (2) contains every operation that can read the source, so no operation after o_β reads it.*

Proof sketch. By induction over graph order, the union in Eq. (2) keeps r in $S(v)$ for every value v that may alias the source, and the read rule places each operation consuming such a value in Q_r. □

The runtime further requires that each reader's completion event follow its source reads, that mappings stay valid, and that identities stay unique while referenced. Under these conditions, enrolling readers before submission (Algorithm 1), returning a borrow only when Eq. (4) holds, and overwriting only after all bits clear preserve the source until every reader finishes, while results keep the identity captured at acceptance.

V. Evaluation

We ask what releasing at the last read gains and costs (Sections V-B–V-D), whether REALBIM keeps every reader on its frame as reads are reordered and models change (Section V-E), and what its checks cost (Section V-F). The hand-placed baselines release at the same point as REALBIM and so match its delivery by design: the throughput gain comes from where the borrow ends. Deriving that point pays off in safety, which Section V-E measures, including a residual read that a boundary placed by hand after the first read would miss.

A. Setup

Platform. We evaluate cross-process reuse on one NVIDIA A100 PCIe 40 GB GPU. The producer, four recipients, and an output sink are separate processes that communicate through multiprocessing queues and share storage and events through CUDA IPC. To isolate reuse and admission from feature extraction, the producer replays saved BEV features. The runtime uses PyTorch 2.1.2, CUDA 12.1, and TensorRT 10.3. Without NVIDIA's Multi-Process Service (MPS), recipient kernels take turns on the GPU rather than running concurrently [18]. Records, validators, and analysis scripts are public.[1]

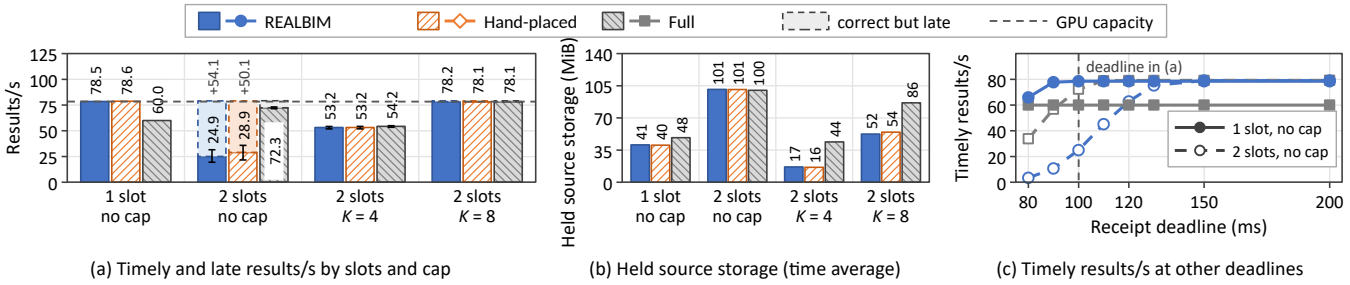
Workload. Graph B is BEVFusion's functional TransFusion head [4], [19], exported with 287 operations, and engine A, the stand-in for a compiled second head, runs the head's shared convolution, ReLU, and global average pooling in TensorRT (Section II). The producer publishes precomputed 63.3 MiB feature maps from three fixed nuScenes-mini frames [5]. Four recipients run identical compositions in the sweeps; Section V-D varies their components. The correctness, model-change, and cost tests use a ResNet-18 tail [13]: its residual stages and classifier read saved first-layer activations.

Policies. All policies share the pool, workloads, queues, and admission settings. *REALBIM* uses both layers, *Full* is full retention, and *Clone-on-accept* gives each recipient a private input copy when it accepts a request. *Hand-placed* is a separate runtime that enrolls both readers, releases at a boundary set by hand, and checks code and model digests; it tests early reuse independently of our code (Sections V-B and V-C). *REALBIM (hand-placed)* is REALBIM with its boundary set by hand instead of derived, so the two differ only in where the boundary comes from (Section V-D).

Metrics. The producer schedules frames at rate λ, in Hz. Each frame should yield four *results*, one output bundle per recipient, so the offered load is 4λ results/s. A result is *timely* if it passes source-identity and reference-output checks and reaches the output sink within 100 ms of the frame's planned arrival, including device-to-host transfer and IPC; Fig. 7c rescores the slot runs at other deadlines. *Receipt latency* runs from planned arrival to sink receipt, and *slot hold time* from commit to the last borrow return. Section V-D uses the *complete-frame timely fraction*: the share of scheduled frames, refused ones included, with all four results timely.

Runs and statistics. Table II lists the experiments. The producer drops frames rather than blocking, so every policy sees the same offered load. In the sweeps, each recipient holds at most two *pending* requests (queued, not yet accepted) and four *in flight* (accepted, results not yet delivered). A run has 2 s of warm-up, 12 s of measured arrivals, and a complete drain, in randomized order. Because these runs report ratios

[1] https://github.com/assasin831/readseal-artifact/tree/ispa2026-r19.



(a) Timely and late results/s by slots and cap

(b) Held source storage (time average)

(c) Timely results/s at other deadlines

Fig. 7. **One REALBIM slot matches full retention with two slots and an admission cap** (30 Hz; six runs per condition and policy). K caps outstanding recipient requests. (a) Solid: timely results; dashed: correct but late; whiskers: 95% intervals. (b) Time-averaged source storage held by unreturned borrows. (c) The same runs re-scored at other deadlines.

TABLE II
EXPERIMENTS. RUNS ARE PER POLICY AND CONDITION; K IS THE ADMISSION CAP.

Experiment	Policies	Rate (Hz)	Slots, K	Runs	Metric	Where
Rate sweep	R H F	10–40	1, off	6	results/s, ms	Fig. 6a–c
Late read	R H F	30	1, off	6	results/s	Fig. 6d
Slot study	R H F	30	1–2, off/4/8	6	results/s, MiB	Fig. 7
Campaigns	R Rh F C	30	1, off	6 (+12)	% frames, p99	Fig. 8
Vs. copies	R C	20–40	1, off	168	results/s, p99	§V-D
Safety	R H Rh S	—	—	6/96/22/16	identity, outputs	§V-E
Cost	R H	—	—	8×50	ms per call	§V-F

Policies: R REALBIM, H Hand-placed, Rh REALBIM (hand-placed), F Full, C Clone-on-accept, S stream-only.

and paired differences, brackets give pointwise 95% intervals from 20,000 paired whole-run bootstrap draws. The campaigns report per-policy fractions with pointwise 95% t intervals; they use queues of eight, 10 s of warm-up, and 30 s of measurement.

B. Early Reuse Raises Delivery Under Load

Early reuse pays off once the frame period drops below Full's hold time (Fig. 6a). Because the recipients take turns on the GPU and Full's 51 ms hold spans nearly all of their computation, the GPU can finish at most about four results per 51 ms, or 78 results/s (dashed line). From 20 Hz on, the offered load reaches this capacity, so each frame Full refuses leaves GPU time unused. At 20 Hz, where Full accepts only every other frame, REALBIM delivers 1.95× Full's timely results, and at 25–40 Hz 1.31–1.50× as many, near capacity. REALBIM stays within 0.2 results/s of Hand-placed throughout.

The gain comes from a shorter hold: Full holds the slot for about 51 ms, whereas REALBIM returns it once the last recipient's first operation has run, in 30–34 ms, since the recipients take turns on the GPU (Fig. 6c). The shorter hold lets the next frame enter while private computation continues, which fills the idle gaps of Fig. 4. The dip at 25 Hz comes from periodic arrivals. The 40 ms period is shorter than REALBIM's longer holds (95th percentile 44 ms), so after each refusal the slot waits a full period for the next arrival, which comes sooner at 30 and 40 Hz.

The extra admitted work queues on the GPU, which costs latency. At 20–40 Hz, REALBIM's mean latency is 8–11 ms higher than Full's and its per-run 95th percentile 15–19 ms higher (Fig. 6b). At 30 Hz REALBIM's delivery lead holds for deadlines from 100 to 200 ms and narrows at 80 ms, where more of this queued work arrives late (Fig. 7c).

The gain depends on where the last read falls. Adding a read at operation 288 of 289 (Fig. 5, Add late read) leaves almost nothing to overlap: REALBIM's hold reaches Full's, and all three policies deliver 60.0 results/s at 30 Hz (Fig. 6d).

C. One Slot Matches Two, and Two Need a Cap

A second slot is the usual remedy for Full's refusals (Fig. 7a). At 30 Hz, two-slot Full with $K{=}8$ delivers 78.1 results/s, and one-slot REALBIM delivers 78.5, one 64 MiB buffer fewer per shared tensor. The second slot also makes the cap decisive: $K{=}4$ holds every policy to 53–54 results/s, and without a cap work arrives late.

With one slot, the hold time in Eq. (1) paces REALBIM's admissions. A second slot removes that pacing and lets requests queue before the GPU can start them. As with bufferbloat in networks, the longer queue adds delay without adding throughput: without a cap, two-slot REALBIM delivers only 24.9 timely results/s, and another 54.1 arrive correct but late. Full's longer hold still throttles its arrivals, so it delivers 72.3. Hand-placed behaves like REALBIM, and looser deadlines recover both (Fig. 7c).

At $K{=}8$, all three policies reach similar delivery (Fig. 7a), but REALBIM returns borrows sooner. The source storage held by borrows therefore averages 52.3 MiB under REALBIM against 86.4 MiB under Full (Fig. 7b), memory that an allocator could lend to other work.

D. Jitter, Mixed Recipients, and Copying

In the campaigns (Fig. 8), arrivals are periodic or displaced by up to 0.8 of a period (*jitter*), and recipients are either *identical* copies of the composition or *mixed*: the native engine alone, the exported head alone, their composition, and a composition whose last read is at the graph's end.

With identical recipients, REALBIM completes 65.5% of frames on time under periodic arrivals, i.e., 78.6 of the 120 results/s offered, the GPU's capacity (Section V-B), and 63.5% with jitter, about 15.5 percentage points above Full in both cases. With mixed recipients, the recipient that reads at the end holds the slot for all four, and REALBIM stays less than one point above Full (Fig. 8a). Jitter raises mixed-recipient delivery from 50.3% to 58.5% for REALBIM and from 50.0%

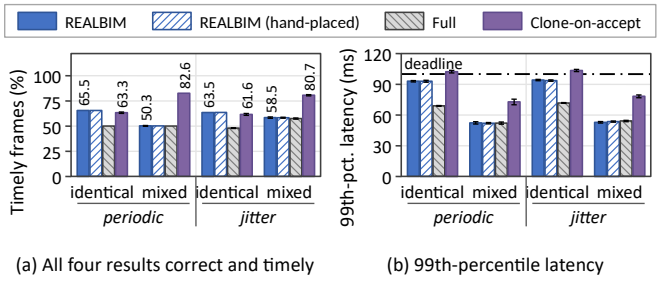


Fig. 8. **Complete-frame delivery and tail latency with jitter and mixed recipients** (30 Hz, one slot, no pipeline-wide cap; six runs per bar; whiskers: 95% t intervals). (a) Share of the 900 scheduled frames per run whose four results are all correct and timely. (b) Per-run 99th-percentile latency of frames whose four results are all correct. *Identical*: four copies of the composition; *mixed*: four component profiles, one with its last read at the graph's end.

to 57.5% for Full, as irregular gaps let more frames past that recipient's hold. Across both campaigns, including 12 runs in which every recipient reads at the end, REALBIM (hand-placed) stays within 0.13 percentage points of REALBIM.

Clone-on-accept returns its borrow when the copy completes, however long computation then reads the private copy. With identical recipients it completes about two points fewer frames on time than REALBIM. With mixed recipients it completes 22–32 points more, because no computation holds the slot, at a 256 MiB higher sampled device peak and 20–25 ms more 99th-percentile latency.

The 168-run comparison agrees. With early last reads, REALBIM delivers 0.4–0.9 results/s more than Clone-on-accept at 20, 30, and 40 Hz, and 2.75 [2.47, 3.03] fewer at its 25 Hz dip. With every last read at the end, at 30 Hz, Clone-on-accept delivers 76.4 results/s to REALBIM's 60.0 (paired gain 16.39 [16.06, 16.72]). It also raises mean per-run 99th-percentile latency from 68.7 to 101.1 ms and device peak by 256 MiB.

E. Safety Across Reordering and Model Changes

We first reorder reads on purpose. In six controlled tests, engine A's input-consumed event completes before graph B submits its reads. Under stream-only release, B's outputs matched the replacement frame in every test, because recording stream use cannot cover reads that have not been submitted [15]; under REALBIM and Hand-placed, they matched the admitted frame. Under load, all 109,592 results delivered by REALBIM, Hand-placed, and Full in the 144 rate and late-read runs carried their admitted identity and matched reference outputs.

We then change the models. In 96 ordered-overwrite cases (eight graph snapshots per model, three inputs, and two methods), the source was overwritten as soon as the regenerated boundary had completed, and the outputs matched the unsplit model's exactly. They include the ResNet-18 tail, whose derived boundary sits at the residual read at operation 12 (Fig. 5, right). A boundary placed by hand after the first read would have freed the source under that read, unnoticed. Binding checks rejected all 22 attempted substitutions of plan, code, globals, model state, or input before the affected stage ran.

Finally, we replay 16 controlled transitions: the 14 graph edits of Fig. 5, one folding of batch normalization into the preceding convolution, and a public ResNet-18 to ResNet-50 substitution, whose binding check runs on the CPU. Some move the last source read; others only renumber it or leave it in place. Binding checks reject every outdated hand-placed plan, which then waits for a person to move its boundary. ReadSeal rebuilds every updated plan in 0.08–0.21 s on the CPU, with no annotations. In the mixed-recipient campaign, adding a read at the end made the old boundary of REALBIM (hand-placed) fail its binding check until the boundary was moved; all 24 update cases then gave exact outputs.

F. Execution Cost and Coverage

An isolated-process test times recipient acceptance to output dispatch (8 process repetitions of 50 randomized invocations per method; three inputs; both models with and without a read added at the end). ReadSeal's checks add 0.8–0.9 ms per invocation, mostly at acceptance and stage submission: 8% over Hand-placed for the base head (11.8 ms against 10.9 ms) and 22–23% for the lighter ResNet tail. In the pipelines above this cost overlaps GPU work; even at 10 and 15 Hz, REALBIM's mean receipt latency is within 0.4 ms of Hand-placed's. A static survey of the tails of 14 torchvision models accepts nine, with boundaries in their first 1–12 operations, and rejects five for unsupported operators.

VI. Discussion

Borrow or copy. Where the last read falls decides between borrowing and copying. With early last reads, a borrow ends about when a copy would, so borrowing delivers as much without the copies' memory and bandwidth. When a reader's last read comes at the end of its graph, copying delivers more (Fig. 6d; Section V-D). Because REALBIM derives every boundary before the reader runs, the choice can follow from the plan rather than from measurement.

Beyond the A100. The delivery gain depends on how the hold compares with the frame period (Eq. (1)), not on memory capacity. A shared-DRAM SoC changes the price of the alternatives. There, a second slot or a private copy takes memory and bandwidth from the rest of the stack [6]; on our 40 GB card, a 64 MiB slot is small beside a 5 GiB device peak. We therefore expect borrowing to save more on an SoC.

Limitations. Our measurements come from one A100 without MPS, where recipients take turns on the GPU [20], [21], and from replayed features rather than a ROS 2 stack with its BEV front end. Our fixed slot pool does not yet lend the storage that early returns free, so device peak is the same for every policy (5,078 MiB with one slot, 5,142 MiB with two). The six ordered tests show that stream-only release can compute on the wrong frame, not how often. The prototype runs declared topologies with single-stream stages. A slow recipient holds the slot for everyone (Section V-D), and a crashed one until recovery stops its GPU work.

VII. Related Work

Zero-copy IPC and cross-process lifetime. Shared-memory transports let ROS 2 subscribers read publisher-owned messages

in place [8], [9]. Hazcat and ROS 2's `rosidl::Buffer` backends carry zero-copy into device memory [10], [11], and NvSciStream lets each consumer return a buffer on its own but leaves where to return it to the application [22]. Agnocast keeps a message until every subscriber has dropped its reference, tracked by `ipc_shared_ptr` with one bit per subscriber [12]. BIM uses the same kind of per-recipient bits, but GPU completion events, not reference drops, clear them, and results carry frame identities. That change lets ReadSeal end a borrow at a derived, checked last read.

Timing and GPU sharing. Real-time analyses of ROS 2 bound chain latency, data age, and the time disparity of fused messages [3], assuming that each message keeps its data and timestamp until consumed, which REALBIM guarantees for shared GPU tensors. Multi-tenant GPU systems preempt or interleave co-located models [20], [21] or schedule around SoC memory contention [6], but manage memory that one runtime allocates, not tensors shared across processes.

Last use and reclamation. Memory planners and borrow checkers track lifetimes, also over GPU memory, but see every use in one program [23], [24]; a producer sees neither its recipients' programs nor their unsubmitted reads. Read-copy-update (RCU) [25] reuses memory once no reader can hold it but protects only readers that have announced themselves. BIM's commit (Section III) instead opens every recipient's read-side section, the interval in which it may read, on its behalf, and ReadSeal closes it at the last read. ReadSeal's bindings, like PyTorch 2's guards [17], reject execution whose assumptions changed, but where a failed guard recompiles, a failed binding refuses to run, because another process relies on the boundary.

VIII. Conclusion

Sharing one GPU buffer among processes saves the memory and bandwidth of private copies, but only if the producer knows when it may overwrite it. The answer lies in separating two lifetimes: storage is needed until the last read, often a model's first operation, while identity must last until every result is delivered. Full retention, copying, and early reuse then differ only in where the borrow ends, and the last read is the earliest safe point.

REALBIM makes that point usable. BIM protects every recipient from publication on, including readers that have not submitted GPU work; ReadSeal derives each reader's last read and refuses to run if the code, model, or input layout has changed. With early last reads on an A100 at 25–40 Hz, this delivered 1.31–1.50× the timely results of full retention, and one slot matched two under full retention. No reader computed on the wrong frame, and every replayed model change was re-planned within 0.21 s.

Known before a reader runs, the boundary also tells a system which readers should borrow and which should copy. For real-time analyses of ROS 2 pipelines, REALBIM provides the property they assume: each message keeps its data and timestamp until consumed. Next, we will integrate REALBIM with ROS 2's CUDA buffer backend and NvSciStream, measure it on shared-DRAM SoCs, support concurrent recipients, and make that choice automatically.

References

[1] S. Kato *et al.*, "Autoware on board: Enabling autonomous vehicles with embedded systems," in *Proc. ACM/IEEE ICCPS*, 2018, pp. 287–296.

[2] S. Macenski *et al.*, "Robot Operating System 2: Design, architecture, and uses in the wild," *Sci. Robot.*, vol. 7, no. 66, p. eabm6074, 2022.

[3] D. Casini *et al.*, "A survey of real-time support, analysis, and advancements in ROS 2," *Leibniz Trans. Embedded Syst.*, vol. 11, no. 1, pp. 1:1–1:37, 2026.

[4] Z. Liu *et al.*, "BEVFusion: Multi-task multi-sensor fusion with unified bird's-eye view representation," in *Proc. IEEE ICRA*, 2023, pp. 2774–2781.

[5] H. Caesar *et al.*, "nuScenes: A multimodal dataset for autonomous driving," in *Proc. IEEE/CVF CVPR*, 2020, pp. 11 621–11 631.

[6] I. Dagli and M. E. Belviranli, "Shared memory-contention-aware concurrent DNN execution for diversely heterogeneous system-on-chips," in *Proc. ACM PPoPP*, 2024, pp. 243–256.

[7] NVIDIA, "Jetson AGX Orin series data sheet," DS-10662-001, 2023.

[8] T. Ishikawa-Aso and S. Kato, "ROS 2 Agnocast: Supporting unsized message types for true zero-copy publish/subscribe IPC," in *Proc. IEEE ISORC*, 2025, pp. 1–10.

[9] X. Luo *et al.*, "Flexible zero-copy IPC for processing chains in ROS 2," *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, 2025, early access.

[10] O. Bell, C. Gill, and X. Zhang, "Hardware acceleration with zero-copy memory management for heterogeneous computing," in *Proc. IEEE RTCSA*, 2023, pp. 28–37.

[11] Open Robotics, "ROS 2 `cuda_buffer_backend`," https://github.com/ros2/rosidl_buffer_backends, 2026.

[12] T. Ishikawa-Aso *et al.*, "`ipc_shared_ptr`: A publish/subscribe-aware smart pointer for cross-process object lifetime management," in *Proc. IEEE ISORC*, 2026, arXiv:2605.04226.

[13] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE CVPR*, 2016, pp. 770–778.

[14] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Proc. NeurIPS*, 2019, pp. 8024–8035.

[15] PyTorch Contributors, "`torch.Tensor.record_stream` (PyTorch 2.1 documentation)," https://pytorch.org/docs/2.1/generated/torch.Tensor.record_stream.html, 2023.

[16] NVIDIA, "TensorRT 10.3 runtime API: Input-consumed events," https://github.com/NVIDIA/TensorRT/blob/release/10.3/include/NvInferRuntime.h, 2024.

[17] J. Ansel *et al.*, "PyTorch 2: Faster machine learning through dynamic Python bytecode transformation and graph compilation," in *Proc. ACM ASPLOS*, 2024, pp. 929–947.

[18] J. Bakita and J. H. Anderson, "Demystifying NVIDIA GPU internals to enable reliable GPU management," in *Proc. IEEE RTAS*, 2024, pp. 294–305.

[19] X. Bai *et al.*, "TransFusion: Robust LiDAR-camera fusion for 3D object detection with transformers," in *Proc. IEEE/CVF CVPR*, 2022, pp. 1090–1099.

[20] M. Han, H. Zhang, R. Chen, and H. Chen, "Microsecond-scale preemption for concurrent GPU-accelerated DNN inferences," in *Proc. USENIX OSDI*, 2022, pp. 539–558.

[21] F. Strati, X. Ma, and A. Klimovic, "Orion: Interference-aware, fine-grained GPU sharing for ML applications," in *Proc. ACM EuroSys*, 2024, pp. 1075–1092.

[22] NVIDIA, "NvSciStream API reference (DRIVE OS 6.0)," https://developer.nvidia.com/docs/drive/drive-os/6.0.8/public/drive-os-linux-sdk/api_reference/group__nvsci__stream__apis.html.

[23] M. Maas, U. Beaugnon, A. Chauhan, and B. Ilbeyi, "TelaMalloc: Efficient on-chip memory allocation for production machine learning accelerators," in *Proc. ACM ASPLOS*, 2023, pp. 123–137.

[24] B. Köpcke, S. Gorlatch, and M. Steuwer, "Descend: A safe GPU systems programming language," *Proc. ACM Program. Lang.*, vol. 8, no. PLDI, pp. 841–864, 2024.

[25] P. E. McKenney, J. Fernandes, S. Boyd-Wickizer, and J. Walpole, "RCU usage in the Linux kernel: Eighteen years later," *ACM SIGOPS Oper. Syst. Rev.*, vol. 54, no. 1, pp. 47–63, 2020.